EN683 NPU 실시간 탐지

문제점과 해결 방안 · 9월 실험 기록과 벤더 SDK 문서를 다시 맞춰 보며

EN683에는 2.7 TOPS NPU가 있다. 9월 실험에서 NPU 연산 자체는 한 장에 약 37 ms로 잘 돌았지만, 카메라 영상을 NPU에 넣는 라이브 경로가 모두 막혀 JPEG 스냅샷을 거치는 우회로를 썼고, 그 결과는 **약 1.5 fps**였다. 그래서 펌웨어 릴리스에서는 NPU를 뺐다. 이 문서는 무엇이 막혔는지 정리하고, 벤더 SDK 안내와 우리 설정을 대조해 찾은 원인 가설과 해결 절차를 적는다.

2026년 10월 20일 작성. 대상: EN683 EVK + Sony IMX675, 벤더 SDK와 예제 모델 `od6class`(사람·차량 6종 탐지, 입력 832×480). 4.1절의 원인은 문서 대조로 얻은 가설이고 아직 보드에서 확인하지 않았다. 벤더 SDK 문서는 공개 자료가 아니므로 SDK 고유의 파라미터 이름, 함수 이름, 명령은 일반적인 표현으로 바꿔 적었다.

1. 요약

문제	원인	해결 방안	상태
① UYV 경로 거부(`0x322`)	ISP 설정에서 NPU용 버퍼가 꺼져 있음	버퍼를 켜고 ISP FW 재빌드	가설, 미확인
② VCAP 경로 커널 패닉	벤더 드라이버의 인터럽트 핸들러가 포인터를 검사하지 않음	이 경로는 쓰지 않음. 벤더 수정 요청	벤더 몫
③ 스냅샷 우회 1.5 fps	프레임마다 JPEG 왕복, CPU 디코드, 프로세스 생성	①이 풀리면 버림	①에 달림

핵심 NPU는 느리지 않다. 느린 것은 NPU 앞의 입력 경로다. 정식 경로(UYV)를 막고 있던 것은 드라이버가 아니라 우리 ISP 설정일 수 있다.

2. 배경: EN683 NPU와 SDK

2.1 공개된 사양

항목	값 (아이닉스 홈페이지)
연산	2.7 TOPS, INT8
연산자	지원 연산자가 정해져 있고, 그 연산자만으로 된 모델만 올라간다
입력 모델	ONNX(권장), tflite
양자화	가중치는 채널별 대칭, 활성화는 텐서별 비대칭. PTQ만 지원(QAT 없음)
도구	컴파일러, PC에서 도는 시뮬레이터(Python 패키지), Model Zoo
지원 모델 예	MobileNetV2, ResNet, EfficientNet-Lite, OSNet, YOLOv3–v9, YOLOv8-seg

2.2 SDK 흐름

모델은 작업용 PC에서 학습하고 컴파일한 뒤, 바이너리만 보드로 옮겨 실행한다.

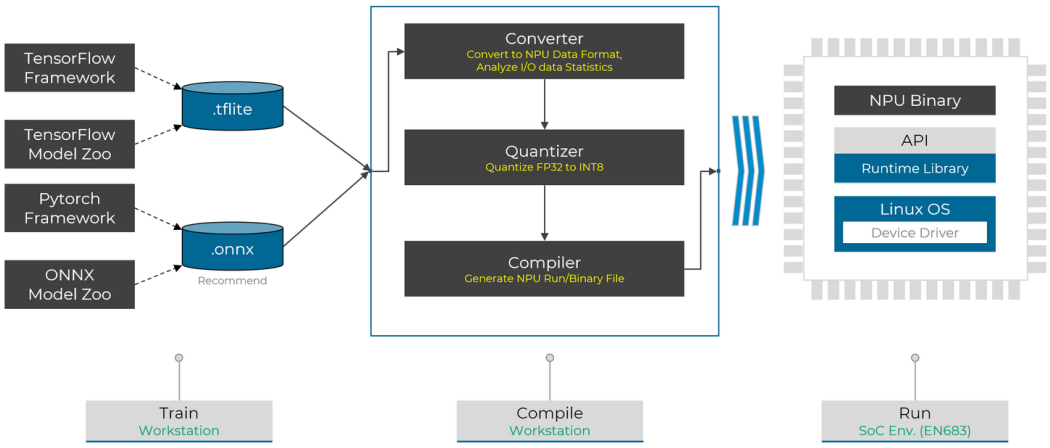


그림 1. EN683 NPU SDK 흐름(아이닉스 홈페이지). 이 문서의 문제는 모두 오른쪽 Run 단계, 그중에서도 NPU에 영상을 넣는 입력 부분에 있다.

보드에서 도는 벤더 예제 실행기는 입력을 받는 방법이 셋이다. 이미지 파일, **UYV**(ISP가 NPU용으로 따로 리사이즈해 쓰는 RGB 버퍼), **VCAP**(인코더가 쓰는 영상 채널의 NV12를 함께 읽기)이다. 파일을 주지 않으면 UYV를 쓴다.

3. 문제점

3.1 세 입력 경로와 그 결과

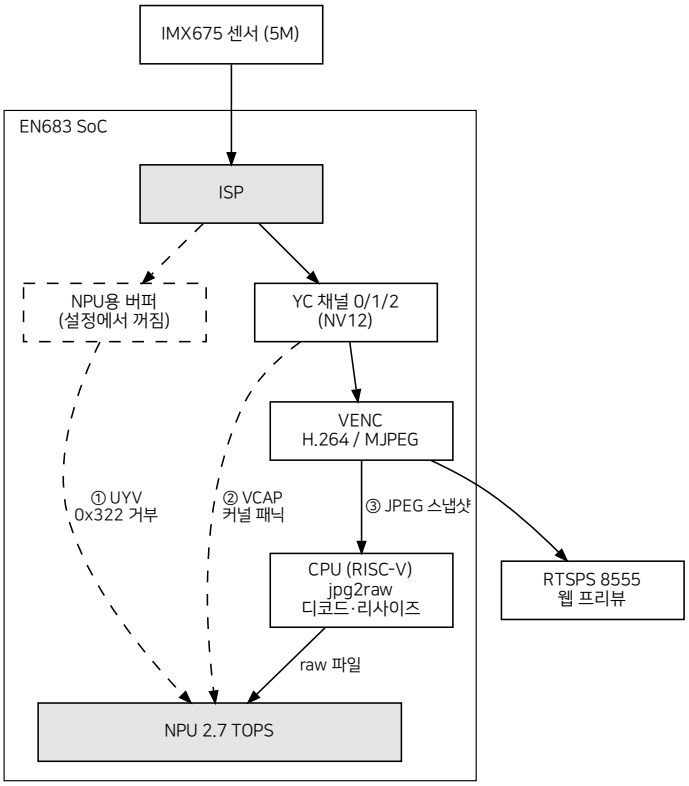


그림 2. EN683 영상 경로. 점선 ①②가 NPU 정식 입력 경로이고 둘 다 실패했다. 실선 ③이 실제로 쓴 우회로다.

경로	결과	증상
① UYV	실패	캡처 초기화가 링크 생성 오류(`0x322`)로 거부됨. 인코더를 모두 멈춰도 같다
② VCAP	실패 (치명)	캡처 시작 직후 첫 프레임 인터럽트에서 커널 패닉. 보드 정지, 재부팅 필요
③ 스냅샷	동작	사람 탐지 성공(score 0.638), 그러나 약 1.5 fps

3.2 ② VCAP: 인터럽트 문맥의 커널 패닉

시리얼 콘솔에서 받은 oops를 요약하면 이렇다. 프레임 완료 인터럽트를 처리하던 벤더 미디어 드라이버가 스트림 버퍼 서술자를 따라가다가, 초기화되지 않은 포인터가 가리키는 주소를 읽었다(`cause 0x0d`, load page fault). 인터럽트 문맥이라 커널은 복구하지 못하고 `Fatal exception in interrupt`로 멈췄다. 실행 중이던 프로세스는 `swapper/0`, 즉 유휴 상태의 CPU였다. 사용자 프로그램이 아니라 드라이버 안에서 난 결함이라는 뜻이다.

3.3 ③ 스냅샷 우회: 숫자로 본 차이

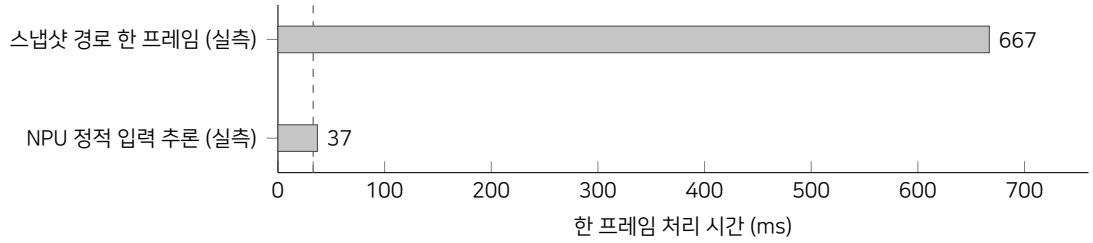


그림 3. 스냅샷 경로의 667 ms는 기록된 1.5 fps를 환산한 값이다. 점선은 30 fps(33 ms) 선이다. 같은 NPU가 정적 입력으로는 27 fps를 낸다.

NPU 추론은 37 ms인데 한 프레임에 약 667 ms가 걸렸다. 나머지 약 630 ms(전체의 94%)는 NPU 바깥에서 쓰였다. 구간별 시간은 따로 재지 않았으므로, 어느 단계가 가장 큰지는 2단계 측정에서 확인한다.

핵심 정식 입력 경로 둘은 막혔고, 우회로는 NPU 성능의 1/18만 쓴다.

4. 원인 분석

4.1 ① UYV: 버퍼가 꺼져 있었다 (가설)

벤더 SDK의 NPU 시작 안내에는 "NPU는 ISP와 함께 돌아야 하므로 ISP 펌웨어 파라미터를 고쳐 다시 빌드하라"는 단계가 있다. 그 단계에서 고치라는 항목을 우리 설정과 나란히 놓으면 다음과 같다.

ISP 파라미터	우리 값	안내 값
NPU용 버퍼 페이지 수	0 (쓰기 꺼짐)	2
NPU용 버퍼 가로 크기	0 (원본 크기)	832
NPU용 버퍼 세로 크기	0 (원본 크기)	480
버퍼 입력 소스	센서	같음

8월에 센서를 붙일 때 센서 종류와 해상도 항목만 고쳤고 NPU용 버퍼 항목은 손대지 않았다. 즉 ISP가 NPU용 버퍼를 아예 만들지 않는 펌웨어로 UYV 캡처를 시도한 것이다. 이 가설을 받치는 근거는 셋이다.

1. 파라미터 설명에 따르면 페이지 수 0은 "버퍼 쓰기 꺼짐"이다.
2. 버그리포트 기록: 인코더와 감시 서비스를 **모두 멈춰도** `0x322`가 났다. 채널 부족이 아니라 경로 자체가 구성되지 않았다는 뜻이다.
3. `0x322`는 패닉이 아니라 링크 생성 단계의 **거부**다. 없는 버퍼에 링크를 만들려 했다면 나올 법한 반응이다.

반대 근거도 적어 둔다. `0x322`의 정확한 뜻은 문서에 없다. 그래서 1단계를 하고도 같은 오류가 나면 이 가설은 버린다.

4.2 ② VCAP: 우리 쪽에서 고칠 수 없는 결함

인터럽트 문맥에서는 페이지 폴트를 복구할 수 없다. 드라이버는 포인터를 쓰기 전에 검사해야 하고, 지원하지 않는 구성이면 붙기 전에 오류로 거부해야 한다. 소스가 없는 벤더 모듈이므로 우리는 이 경로를 **쓰지 않**

는 쪽을 택하고, 수정은 버그리포트로 요청한다. VCAP은 인코더와 같은 영상 채널을 나눠 읽는 구조라 원래도 충돌 위험이 크다.

4.3 ③ 스냅샷: 왜 느린가

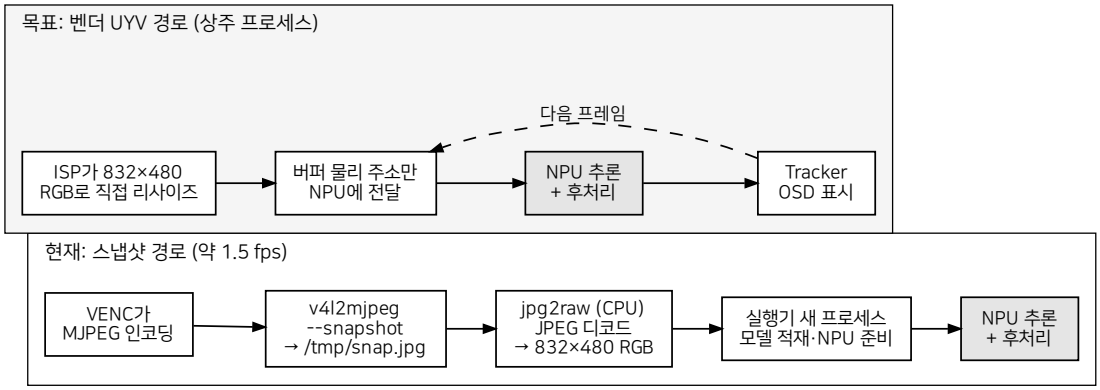


그림 4. 한 프레임을 처리하는 단계. 위가 벤더 정식 경로, 아래가 지금 쓰는 스냅샷 경로다.

스냅샷 경로는 매 프레임 다음 일을 되풀이한다.

- 인코더가 만든 JPEG을 파일로 받는다(압축).
- CPU가 그 JPEG을 다시 풀고 832×480으로 줄인다(압축 해제와 리사이즈).
- 실행기를 **새 프로세스**로 띄워 모델을 읽고 NPU를 준비한 뒤 한 장만 추론하고 끝낸다.

정식 경로에서는 이 일들을 하드웨어가 대신하거나 아예 하지 않는다. 리사이즈와 색 변환은 ISP가 하고, 실행기는 상주하면서 버퍼의 물리 주소만 NPU에 넘긴다. 프레임을 사용자 메모리로 복사하는 일도 없다.

핵심 스냅샷 경로는 하드웨어가 할 일을 CPU가 하고, 상주해야 할 프로세스를 매번 새로 띄운다. 고칠 대상은 모델이 아니라 경로다.

5. 해결 방안

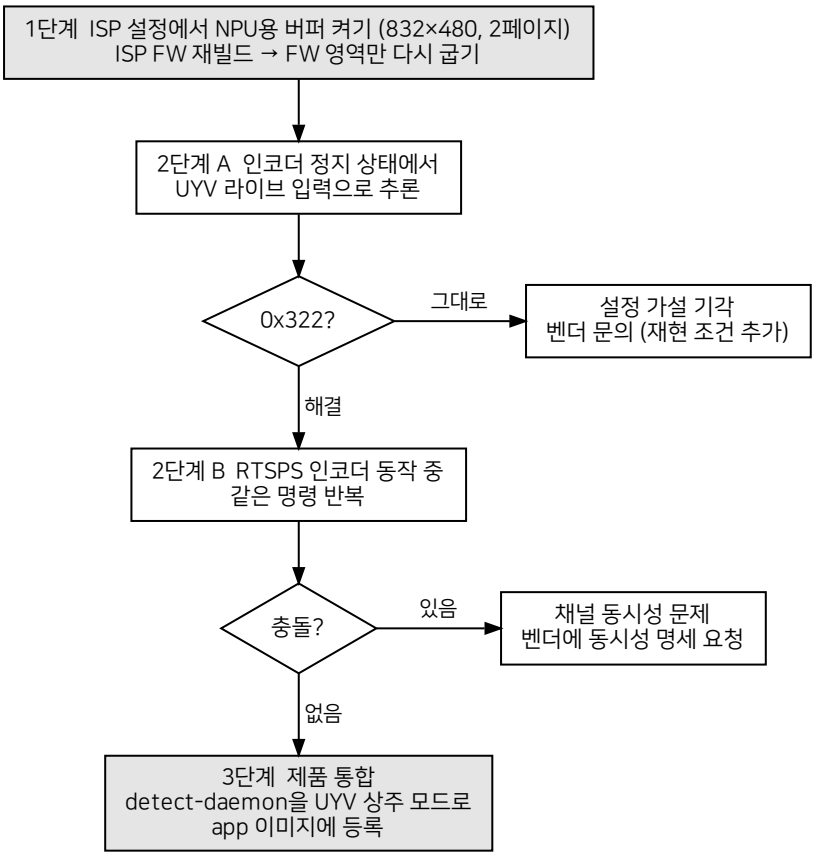


그림 5. 해결 절차. 각 갈림길에서 결과에 따라 다음 행동이 정해진다.

5.1 1단계: ISP에서 NPU용 버퍼 켜기

1. 지금 플래시에 있는 ISP 펌웨어를 백업한다.
2. ISP 파라미터 파일에서 NPU용 버퍼를 켠다. 페이지 2, 크기는 모델 입력과 같은 832×480(가로는 16의 배수). 우리 BSP 오버레이에도 같은 값을 넣어 다음 빌드에서 사라지지 않게 한다.
3. ISP 펌웨어만 다시 빌드한다. 빌드 과정에서 파라미터 값이 펌웨어 바이너리 안에 들어간다.
4. U-Boot에서 **ISP 펌웨어 영역만** 다시 굽는다.

ISP 펌웨어 영역만 바꾸기 때문에 앱 영역과 `/data`(계정, 키, 감사기록)는 그대로다. 문제가 생기면 백업본을 다시 굽는다. 굽고 나서 RTSPS와 웹 영상이 전과 같은지 먼저 본다.

5.2 2단계: UYV 라이브 입력으로 측정

이미지를 다시 만들지 않고, 실행기와 모델, NPU 드라이버를 RAM(`/tmp`)에만 올려 시험한다. 9월 실험과 같은 방식이라 재부팅하면 흔적이 남지 않는다. 입력 파일 없이 실행해 UYV 라이브 입력을 쓰고, 300회 추론하면서 실행기의 프로파일러로 구간별 최소, 최대, 평균 시간을 받는다. 두 조건으로 잰다.

조건	확인할 것	통과 기준
A. 인코더 정지	`0x322`가 사라지는가, fps	오류 없음, 20 fps 이상
B. RTSPS 인코더 동작 중	패닉·프레임 손실·인코더 끊김이 없는가	300회 무오류, RTSPS 30 fps 유지

B는 9월에 패닉이 났던 조건과 같으므로 시리얼 콘솔을 켜 둔다. 통과 기준 20 fps는 정적 입력 27 fps에서 ISP 대기와 후처리 몫을 뺀 잠정값이다.

5.3 3단계: 제품에 통합

2단계를 통과하면 탐지 데몬(`detect-daemon.sh`)의 스냅샷 루프를 상주 실행기(UYV 모드)로 바꾼다. SDK가 제공하는 앱 이미지 빌드 명령은 쓰지 않는다. 기본값이 NAND 플래시용이라 우리 NOR 보드에서 마운트되지 않는 이미지가 나오고, NPU 테스트 파일을 모두 앱 영역(약 12 MB)에 넣기 때문이다. 대신 필요한 파일만 앱 디렉터리에 넣고 다음을 갱신한다.

- 파일 권한 목록, 무결성 매니페스트, 프로비저닝 대상 목록
- 부팅 때 적재할 커널 모듈 목록(NPU 드라이버, UYV 캡처 드라이버)
- 이미지는 우리 NOR용 이미지 생성 스크립트로 만든다

6. 위험과 대안

위험	대응
1단계 후에도 `0x322`	가설 기각. 버그리포트에 "버퍼를 켠 뒤에도 거부"를 더해 벤더 기술지원에 문의
NPU용 버퍼로 메모리 사용 증가	832×480×3×2페이지 2.4 MB. 영상용 공유 메모리 안에서 충분할 것으로 본다
인코더와 동시에 쓸 때 충돌	벤더에 동시성 명세(가용 채널 수, 인코딩과 분석 동시 지원 여부) 요청
od6class보다 큰 모델	메모리 맵과 NPU 작업 버퍼 크기를 조정한다(벤더 안내 있음)

7. 정리

1. EN683 NPU는 od6class 한 장을 약 37 ms에 추론한다. 연산 성능은 문제가 아니다.
2. NPU 정식 입력 경로 두 개가 막혔다. UYV는 `0x322`로 거부됐고, VCAP은 커널 패닉을 냈다.
3. 우회로인 스냅샷 경로는 1.5 fps다. 한 프레임 시간의 94%가 NPU 바깥(JPEG 왕복, CPU 디코드, 프로세스 생성)에서 쓰인다.
4. UYV 거부의 원인은 우리 ISP 설정(NPU용 버퍼 꺼짐)일 가능성이 높다. 아직 가설이다.
5. VCAP 패닉은 벤더 드라이버의 결함이다. 이 경로는 쓰지 않고 수정을 요청한다.
6. 해결 절차는 버퍼 켜기 → 인코더 없이 측정 → 인코더와 함께 측정 → 제품 통합이다. ISP 펌웨어 영역만 바꾸므로 언제든 되돌릴 수 있다.